

OptSem-C: Auditing Federated SQL Optimizer Contracts

Haoyi Zhang

Xi'an Jiaotong-Liverpool University
Suzhou, China
hyeliozhang@gmail.com

Xunzhu Tang

University of Luxembourg
Luxembourg, Luxembourg
xunzhu.tang@uni.lu

Huaijin Ran

Nanyang Technological University
Singapore, Singapore
huaijin003@e.ntu.edu.sg

Tegawendé F. Bissyandé

University of Luxembourg
Luxembourg, Luxembourg
tegawende.bissyande@uni.lu

ABSTRACT

Optimizer comparisons often start from compact public labels: pushdown, join, cost based, adaptive, or enabled. Those labels are useful only after the denominator is clear. In federated and cloud SQL systems, a filter or aggregate can be rewritten locally, delegated to a connector, exposed as a service-stage graph, or applied as a runtime filter; treating these public contracts as one row can make a later latency or plan-quality explanation ambiguous. OptSem-C audits that denominator. It encodes each source-supported optimizer statement as a guarded public contract over action, guard, layer, placement, decision time, observability, and evidence support, then tests which distinctions a comparison vocabulary drops. In a finite seven-engine audit with 287 source-linked rules, 4,216 executable SQL probes, and 26 admitted public surfaces, keyword and operator-only vocabularies collapse 254 and 238 unequal public-contract signatures, concentrated in four architectural-boundary pairs; the other 17 engine pairs have no headline collision in this corpus. Public yes/no controls add only six corpus-specific witnesses, a positive boundary result for enablement-style controls rather than a broad danger claim. Placement separates keyword witnesses; optimizer layer separates operator-only witnesses; and layer plus placement separates the 492 primary witnesses plus the six yes/no boundary witnesses inside this declared corpus. OptSem-C is a corpus-specific, replayable public-contract audit for deciding when a comparison vocabulary is specific enough, not a population estimate, a practice survey, a performance benchmark, or a private-engine validation study.

PVLDB Reference Format:

Haoyi Zhang, Huaijin Ran, Xunzhu Tang, and Tegawendé F. Bissyandé. OptSem-C: Auditing Federated SQL Optimizer Contracts. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://doi.org/10.5281/zenodo.21215168>. Archive SHA-256: 39fd8a46dd36600fecdd50e70fe8939d97bde7c5b8062cdce4bceac58b9e18627.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

SQL deliberately hides physical execution choices. That abstraction is right for applications, but a database study often needs to compare the hidden choice itself. A benchmark author may ask whether two engines support pushdown, a system integrator may compare the evidence available in plan output, and a researcher may group systems by join, cost, or adaptivity labels before explaining a performance difference. The same SQL query can then expose different public behavior: a local predicate rewrite, connector-side execution, a service-stage graph, a runtime filter, or only a logical explanation. If a table states only “pushdown=yes”, a later latency result may be comparing different public objects without saying so.

The problem is easiest to see in a concrete audited case. Probe P0009 is a same-connector group-by aggregate over a remote customer table. A keyword view equates BigQuery and Trino under the labels pushdown and explain. The source records do not support the same public contract: BigQuery’s federated record separates source delegation from Cloud SQL aggregate-pushdown blocking, Spanner partial aggregation, and stage-graph evidence, while Trino’s pushdown record separates group-by aggregate pushdown from expression-aggregate blocking and estimated physical-plan evidence. The practical rewrite is simple but consequential: a coarse statement such as “both systems expose pushdown” remains valid as an existence claim, while a portability, plan-evidence, or latency explanation must retain placement, layer, decision time, and observability. Without those fields, the comparison has lost the denominator needed to decide whether two public claims were comparable in the first place.

This is not a claim that an existing paper necessarily reached a wrong conclusion. It is a precondition check: before a paper attributes a difference to an optimizer feature, the compared feature has to be named at the level at which the public systems actually expose it. OptSem-C therefore treats the vocabulary as a measured denominator, not as a survey of all optimizer papers. We audit 26 public comparison surfaces drawn from engine documentation, plan interfaces, and optimizer-control descriptions admitted by the same source policy as the rules. All expose keyword-style or operator-family labels, twelve expose yes/no-style controls, and none exposes the full reference payload used by OptSem-C. The primary contribution is a replayable audit method plus an empirical boundary result: in this corpus, lossy labels fail at federated, cloud-service, and distributed-runtime boundaries, while most local-engine pairs and enablement-style controls do not create headline collisions.

Database research already has precise abstractions for the objects it chooses to study. Relational semantics starts from Codd’s model [20]; Selinger-style optimizers separate access-path search from costing [60]; Volcano exposes rule-driven and parallel query processing [30]; the Volcano optimizer-generator account makes extensible search explicit [32]; and Cascades turns that line into a task-driven framework [31]. Chaudhuri surveys optimizer architecture [14]; Ioannidis surveys cost and cardinality assumptions [38]; and Steinbrunn et al. study join-order search heuristics [63]. Public optimizer behavior is usually compared with much coarser vocabularies. A cell labeled “pushdown” may mean predicate movement inside the engine, partition pruning, aggregation delegation, join delegation, limit delegation, or runtime filtering. A cell labeled “join” may refer to a logical operator, an estimated physical operator, a distributed exchange, or a runtime profile entry. A claim that two engines are “cost based” may hide whether statistics are local, connector-provided, unavailable, or revised during execution. These shortcuts are not harmless summarization: they can create projection-induced contract collisions (a coarse vocabulary declares equality among public contracts whose reference signatures differ).

OptSem-C treats this precision problem as a query-processing systems problem, not as prose cleanup. A *public optimizer contract* is the optimizer behavior an engine exposes through plan interfaces, tuning surfaces, and system descriptions. A source record can support a contract that is not visible in a particular textual plan, and a plan tag can expose behavior without proving the private cost model. The object is therefore not a hidden implementation trace and not a latency measurement. OptSem-C represents each public statement as a guarded contract rule over query features. The rule records the action, the guard under which it applies, where the action is placed, which optimizer layer owns it, when the decision is made, how the behavior is observable, and whether the public contract requires, permits, forbids, or does not specify the action.

Research questions. We ask four finite, auditable questions. RQ1 asks whether public optimizer claims can be encoded as source-linked contracts and executable probes without becoming a latency benchmark. RQ2 asks when the declared comparison vocabularies collapse distinct source-supported contracts inside the admitted surface catalog, and when coarse controls remain benign. RQ3 asks which semantic fields separate those collisions, and which robustness checks delimit the corpus-specific certificate. RQ4 asks whether the projection-audit inner loop and SQL-denominator checks are cheap enough to rerun when the public source denominator changes, rather than trusted as a one-off manual inspection.

The central result is a corpus-specific projection-kernel account of information loss in this seven-engine corpus. Comparing optimizer contracts through a keyword or operator vocabulary applies a predeclared projection to reference signatures: the full public evidence payload inside the declared schema, not an oracle over private optimizer state. If reference signatures disagree but fall into the same projected class, the projection has declared an equivalence that source-supported public signatures do not support. The witness does not prove an optimizer bug or a false performance result. It proves that an unqualified equality claim needs either a narrower scope statement or retained fields before downstream measurements can be interpreted.

OptSem-C turns that denominator question into a finite comparison relation with source-linked counter-witnesses, executable SQL representatives, exact separation certificates, and negative controls. Figure 1 shows the end-to-end boundary. The contract schema, source manifest, projection portfolio, feature-space crosswalk, and retained-field universe are frozen before witness counting; execution checks validate probes but never admit rules or choose fields. Layer and placement are architecture hypotheses from optimizer and federation literature; the replay confirms that this predeclared field pair is the most interpretable basis among the finite safe certificates, not an empirical discovery procedure and not proof of a universal vocabulary. The test is falsifiable: placement alone fails operator-only, layer alone leaves keyword witnesses, and point-learned engine-pair bases fail held-out pairs. Keyword projection declares 2,284 engine-probe equivalences, including 254 collapsed unequal signatures; operator-only declares 2,268, including 238. These 492 primary witnesses occupy four observed engine pairs and 486 distinct probes. The binary yes/no projection contributes only six additional records, counter-evidence to a broad binary-control danger claim. Retaining placement separates keyword witnesses; retaining optimizer layer separates operator-only witnesses; retaining layer and placement also covers the yes/no boundary records within the admitted public-contract corpus.

The robustness checks are designed to block three tempting overclaims. First, the probe suite is a rule-aware coverage test, not an independent population sample, so we do not report statistical generalization beyond the frozen denominator. Second, the four engine-pair hotspots are not hidden: they show that projection risk is concentrated around cloud-service, connector, and distributed-runtime boundaries rather than uniformly spread across all engine pairs. Third, source-removal is an information-deletion stress rather than a new sample: keyword and operator-only remain nonzero after every single-source removal, and the audit separately records retained original witnesses and newly induced witnesses. The sparse yes/no projection is source-sensitive and is reported as a narrow public-control result, not as evidence that every binary feature table is safe. Last, field certificates are corpus certificates: the reported layer-and-placement basis resolves feature-family and engine-family stress splits, while point-learned engine-pair bases fail on held-out pairs and remain a boundary. A field-only stress test over all 256 retained-field vocabularies leaves only 44 unsafe regions, all covered by concrete counter-witnesses; after excluding action-variant identifiers, no singleton semantic field is safe.

We also introduce OptSemBench-C, a contract-probe suite for optimizer-comparison audits. Its denominator is not raw query count or workload runtime. It is coverage of optimizer-relevant feature interactions: source boundary, connector capability, join shape, predicate class, aggregation, statistics need, reuse, distribution, adaptivity, and control surface. The generated probes are executable SQL representatives. The probe space is then cross-checked against DPccp-style join enumeration [51], JOB optimizer evidence [45], later JOB diagnostics [46], exact-cardinality testing [15], progressive optimization [49], mid-query reoptimization [41], adaptive query processing [26], and eddies [7]. We use this audit as feature-space coverage of published optimizer requirements: 90 declared requirements are covered by 71 distinct generated SQL representatives. The check is concrete about optimizer-feature span without

pretending that generated representatives reproduce JOB, TPC-H, or TPC-DS workloads, data distributions, or selectivity effects.

Contributions. C1 defines a public optimizer contract as a query-processing object separate from SQL result equivalence, run-time performance, and private implementation truth; Section 3 gives the guarded rule model and the state-join semantics. C2 turns comparison validity into a finite projection audit: reference signatures define the public evidence relation, projected signatures expose counter-witnesses, and retained-field certificates identify fields that make the observed vocabulary specific enough. C3 contributes OptSemBench-C as a contract-probe denominator for optimizer-comparison audits across seven analytical SQL engines; Section 4 reports source-linked admission, executable probes, feature-space coverage, and SQL-constructability checks. C4 reports corpus-specific positive and negative projection results: 254 keyword witnesses, 238 operator-only witnesses, the six-record yes/no boundary, architecture-readable field bases for those finite records, and cloud replay evidence for the comparison inner loop.

Each central claim is tied to a reproducible record rather than to prose alone. The model section defines the state table and projection obligations; the corpus section exposes source identifiers, line spans, and digest prefixes; the evaluation tables connect SQL execution, feature-space coverage, collision counts, and retained-field certificates to regenerated outputs. The case studies then show how those aggregate certificates change comparison judgments.

2 PRECISION LOSS IN OPTIMIZER COMPARISONS

Figure 2 gives the smallest example. A feature matrix maps several public optimizer contracts into the same label. After the mapping, behaviors with different placement, action kind, and evidence layer become indistinguishable, so a study that treats the label as equality can compare non-equivalent public optimizer behavior. The issue is prior to workload choice: JOB or TPC-H can standardize queries, but they do not say whether two engines expose the same public pushdown, adaptation, or plan-evidence contract.

Representative witness audits. Table 1 gives four small witnesses before the formal model. Each starts from a familiar public label, shows the contract distinction that the label hides, and names the field that separates the comparison. C1 is the running P0009 example: a coarse pushdown/explain label hides BigQuery source delegation, BigQuery aggregate stage evidence, BigQuery connector-specific aggregate-pushdown support and blocking, and Trino connector aggregate-pushdown evidence. C2–C4 cover estimate observability, runtime adaptation, and CTE materialization; the same finite checker covers the 492 primary keyword/operator records and the six yes/no boundary records.

Feature-name collision. A cell such as pushdown=yes may denote predicate movement inside an engine, partition pruning, connector-side predicate delegation, aggregation delegation, join delegation, limit pushdown, Top-N delegation, or runtime filtering. These actions affect different operators and have different guards, placements, decision times, and observable plan effects. Runtime and adaptivity distinctions are visible in Tukwila’s adaptive integration [39], dynamic query plans [21], and Spark SQL adaptive execution [6]. Rewrite and execution-layer distinctions appear in

Table 1: Representative witness audits and safe comparison rewrites. Separators are case-local fields; the reported layer+placement basis covers the primary keyword/operator records and the yes/no boundary records.

Witness	Collapsed public label	Safe rewrite
C1	P0009 links source records where pushdown+explain hides BigQuery source delegation, stage-graph evidence, and aggregate-pushdown support/blocking versus Trino connector aggregate-pushdown evidence.	Keep an existence claim, or retain placement, layer, decision time, and observability before explaining portability or latency.
C2	Sixteen estimate and plan-evidence rules mix local estimates, cloud-service stages, and distributed-worker evidence under one exposed-plan label.	Compare plan evidence only after naming observability and placement; otherwise report a coarse exposed-plan scope.
C3	Sixteen adaptive-execution rules merge compile-time planning with runtime repartition, AQE, and conversion behavior.	Separate compile-time, runtime, and conversion claims before ranking adaptive behavior.
C4	Sixteen CTE rules collapse inline, materialize, reuse, and disable-guard contracts into one materialization label.	State the variant and guard; a single materialization label is safe only for a narrow existence claim.

outerjoin rewrites [29], compiled execution [53], Calcite’s adapter-centered optimizer framework [8], Orca’s modular optimizer architecture [62], and Dremel-style serving trees [50]. A single feature name can therefore erase exactly the fields a comparison needs.

Operator disappearance. A missing native plan node is not a reliable semantics. It may indicate source delegation, logical-plan observability, rewrite, stage-level aggregation, or runtime-only visibility. OptSem-C represents source-delegated execution as a positive contract action rather than as absence. This matters for modern analytical systems whose public surfaces combine Snowflake-style cloud-service stages [23], Redshift’s managed warehouse architecture [35], C-Store column layout [64], row/column execution differences [3], column-oriented design guidance [1], materialization policy [4], compression-coupled execution [2], embedded DuckDB execution [57], vectorized MonetDB/X100 execution [10], HyPer’s compiled in-memory engine [42], Vectorwise vectorized execution [70], Hive’s MapReduce warehouse layer [65], Impala’s Hadoop SQL engine [44], distributed SQL services such as F1 [61], and dataflow-style execution such as Spark RDDs [69].

Architecture ranking. A local engine should not be ranked below a distributed engine merely because broadcast, shuffle, or remote-source delegation is not applicable. Conversely, a federated or cloud engine should not be credited with local optimizer behavior when the public contract exposes delegation or service-side stages. OptSem-C separates non-applicability, documented absence, documented optional behavior, and unspecified evidence.

Observable contract versus hidden implementation. The object of study is not what an engine could do internally. It is what the public contract supports as a comparison claim. This distinction is analogous to the separation between SQL equivalence and optimizer-contract equivalence. Cosette proves SQL equivalences by reducing query semantics to logic [19]; semiring-based decision procedures give another equivalence foundation [18]; and QED

OptSem-C freezes a public-contract denominator before any projection audit

schema, sources, probes, crosswalk, and fields are replay inputs

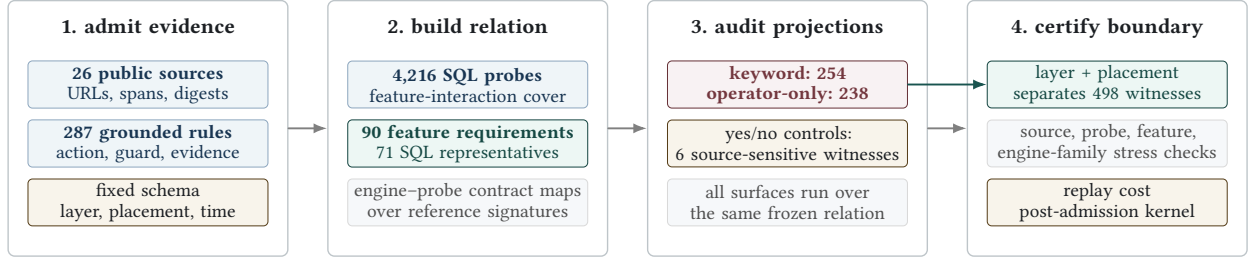


Figure 1: End-to-end replay boundary for OptSem-C. Evidence admission, probe construction, projection comparison, and field certification all operate on the same frozen public-contract relation, so collision counts and separation claims share one finite denominator.

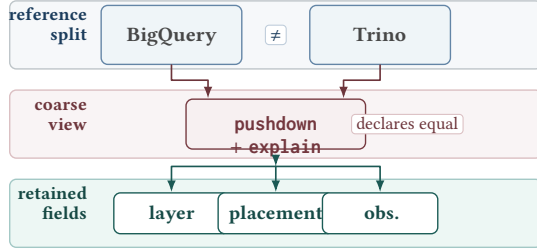


Figure 2: Projection collapse on P0009. The group-by aggregate probe separates BigQuery source/stage evidence from Trino connector aggregate-pushdown evidence under the reference signature; the coarse pushdown+explain vocabulary declares them equal until layer, placement, and observability are retained.

expands practical SQL equivalence checking [66]. Chandra and Merlin characterize conjunctive-query containment [13]; Aho et al. characterize equivalence among relational expressions [5]. OptSem-C asks a different question: whether the public optimizer behavior being compared exposes the same evidence layers and comparison obligations.

These precision losses imply a minimum payload for comparing public optimizer behavior: guard, canonical action, modality, placement, decision time, observability, version, and provenance. The rest of the paper formalizes this payload and measures what happens when a comparison vocabulary projects it away.

3 CONTRACT FORMALIZATION AND NOTATION

A query probe is a SQL skeleton paired with a finite feature vector and validity constraints. Features describe optimizer-relevant structure: join shape and type, predicate class, aggregation, order/limit behavior, source boundary, connector capability, statistics need, reuse structure, distribution trigger, adaptivity trigger, and optimizer control surface. They are not workload-frequency estimates; they are coordinates on the public optimizer decision surface.

The running P0009 witness is a concrete instance before the notation. Under a coarse vocabulary, BigQuery and Trino can both appear as pushdown+explain. The replay ledger binds the pair to BigQuery federated/stage sources and Trino pushdown sources, plus an executable remote GROUP BY COUNT(*) probe. The contract payload keeps the distinctions that the label drops: BigQuery records source delegation, stage-graph observability, Cloud SQL aggregate-pushdown blocking, and Spanner partial aggregate pushdown, while Trino records connector aggregate pushdown, expression-aggregate blocking, and estimated physical-plan evidence. The model below preserves those public distinctions without asserting a private optimizer trace.

The model is task relative. A projection is safe only for the comparison question it declares. If a study deliberately asks the narrow question “does any public source expose a pushdown-like label?”, then a keyword projection may be adequate. If the study compares plan evidence, connector delegation, or runtime behavior, then the same projection can be under-specified. OptSem-C reports this under-specification as a collapsed public-contract comparison, not as an optimizer correctness error.

The retained fields are also task relative. OptSem-C does not learn a field ontology from collisions. It starts from a predeclared architecture vocabulary drawn from optimizer and federation literature, then asks which retained-field subsets separate the finite witness set. The result is therefore confirmation of a stated ontology on this corpus, not discovery of the only correct vocabulary.

DEFINITION 1 (PUBLIC CONTRACT RULE). Let $\mathcal{A}$ be the finite domain of canonical optimizer actions. An action records operator class, action kind, optimizer layer, execution placement, decision time, observability, and variant. The state set $\mathbb{S}$ contains MUST, MAY, MUST_NOT, and UNSPEC. A public contract rule is $r = (\gamma, a, s, v, \epsilon)$, where γ is a guard over query features, $a \in \mathcal{A}$, $s \in \mathbb{S}$, v is the version or time window, and ϵ is evidence provenance. For engine e , C_e is the finite rule set, and $C_e(q) = \{r \in C_e \mid \gamma_r(\phi(q)) = \text{true}\}$ is the rule set applicable to probe q .

The state domain has two components that feature matrices usually conflate. Behaviorally, MUST requires an action, MUST_NOT forbids it, and both MAY and UNSPEC leave it possible. Evidentially,

Table 2: Conflict-aware state join for contract support states. U denotes UNSPEC, N denotes MUST_NOT, and C marks a conflicting evidence merge that must be repaired before the row can enter the admitted corpus.

$\sqcup$	U	MAY	MUST	N
U	U	MAY	MUST	N
MAY	MAY	MAY	MUST	C
MUST	MUST	MUST	MUST	C
N	N	C	C	N

MAY is public support for optional behavior, while UNSPEC is lack of public support. Equivalently, a state is interpreted as (m, u, e) : required behavior, allowed behavior, and evidence support. This is why documented optional pushdown and undocumented implementation possibility are different public contracts.

For each engine-probe pair, OptSem-C constructs a contract map $M_{e,q} : \mathcal{A} \rightarrow \mathbb{S}$, defaulting every action to UNSPEC. Applicable rules update the map with a conflict-aware state join $\sqcup$. The join strengthens unspecified actions when public evidence is found, preserves compatible stronger evidence, and reports contradictions instead of averaging them. The evidence component follows database provenance practice: semiring provenance attaches support to derived facts [34], why/where provenance distinguishes explanation locations [11], provenance surveys separate data from explanation [16], and annotated containment keeps evidence annotations visible in reasoning [33]. The workflow in Figure 1 materializes these maps, tests projected equality against reference inequality, and records retained-field certificates. Table 2 gives the local algebra used inside each map.

DEFINITION 2 (STATE-JOIN ALGEBRA). *The binary operator $\sqcup$ is the complete 4×4 partial join shown in Table 2. It is total over non-conflicting pairs and returns CONFLICT for pairs that simultaneously require and forbid, or support and forbid, the same canonical action. Let $x \leq y$ denote that $x \sqcup y = y$ for non-conflicting states. Then UNSPEC is the least informative state, while MUST and MUST_NOT are incomparable incompatible commitments.*

Here CONFLICT is a merge-failure signal, not a fifth valid contract state; an admitted corpus must resolve it by revising the conflicting evidence record.

LEMMA 1 (MERGE INVARIANTS). *On non-conflicting states, $\sqcup$ is commutative, associative, idempotent, and monotone with respect to $\leq$. Therefore a contract map obtained by joining applicable rules is independent of rule order.*

PROOF. The properties follow by inspection of the finite table. Idempotence holds because every diagonal entry is the input state. Commutativity holds because the table is symmetric. Associativity and monotonicity are checked over the finite non-conflicting triples; the only excluded triples are exactly those containing incompatible positive and negative commitments. Lifting the pointwise operator to maps preserves the properties for every action independently. $\square$

Let $W \subseteq \mathcal{A}$ be an abstract optimizer-behavior world. A map M denotes the world set $\llbracket M \rrbracket = \{W \mid M(a) = \text{MUST} \Rightarrow a \in W, M(a) = \text{MUST_NOT} \Rightarrow a \notin W\}$. The evidential support set is $\text{Supp}(M) = \{a \in \mathcal{A} \mid M(a) \in \{\text{MUST}, \text{MAY}, \text{MUST_NOT}\}\}$, equivalently the actions where $M(a) \neq \text{UNSPEC}$. Since maps default to UNSPEC, every supported non-default entry arises from an admitted public rule. Behavioral containment and evidential support are intentionally separate: two maps can allow the same worlds while differing on whether an action is publicly documented.

DEFINITION 3 (REFERENCE SIGNATURES, KERNELS, AND PROJECTIONS). *The reference evidence-supported signature of M is $S(M) = \{(a, M(a)) \mid a \in \text{Supp}(M)\}$. Reference equivalence is equality of signatures over the full public evidence atom schema. This reference is exact only relative to the admitted public evidence and declared atom fields; it is not a complete private optimizer trace. A projection is a declared map $\pi : \mathcal{A} \times \mathbb{S} \rightarrow \mathcal{B}$ from full evidence atoms to an arbitrary comparison vocabulary $\mathcal{B}$, such as keyword, yes/no, or operator-only labels. The projected signature is the finite image $S_\pi(M) = \{\pi(a, M(a)) \mid a \in \text{Supp}(M)\}$. The projection kernel is the equivalence relation $\mathcal{K}_\pi$ where $M_i \mathcal{K}_\pi M_j$ iff $S_\pi(M_i) = S_\pi(M_j)$. A projection-induced contract collision is a pair in $\mathcal{K}_\pi$ whose reference signatures differ.*

The reference signature is deliberately stronger than many comparison tasks. Thus a collision is a warning about an unqualified equality at projection π : the paper or tool using that vocabulary should either accept the coarser task, state the lost distinctions as out of scope, or retain fields that separate the witness.

For quantitative comparison, exact compatibility is the Jaccard overlap $|S(M_i) \cap S(M_j)| / |S(M_i) \cup S(M_j)|$, and projected compatibility applies the same formula to S_π . The denominator is evidence-supported: undocumented actions do not count as evidence of either equality or inequality.

LEMMA 2 (PUBLIC-CONTRACT SEPARATION). *If $S(M_i) \neq S(M_j)$, then the maps differ in public-contract denotation: either $\llbracket M_i \rrbracket \neq \llbracket M_j \rrbracket$, or their evidence-supported action states differ for at least one action in $\text{Supp}(M_i) \cup \text{Supp}(M_j)$.*

PROOF. If reference signatures differ, some supported action has a state in one map that is absent or different in the other. If the difference changes a MUST or MUST_NOT requirement, the world-set constraints differ. Otherwise the behavioral world set may remain equal, but the public evidence state differs, for example MAY versus UNSPEC. In both cases the denotation differs because an OptSem-C contract is the pair of behavioral worlds and evidence-supported states. $\square$

For finite feature and action domains, constructing $M_{e,q}$ from finite C_e and q is a direct finite evaluation: each guard is a Boolean formula over the probe feature vector, and each applicable rule performs one lookup in the state-join table.

OBSERVATION (PROJECTION-KERNEL COLLISION). *For any projection π , a projection-induced contract collision is exactly the nontrivial part of the kernel $\mathcal{K}_\pi$: M_i and M_j collide iff $S(M_i) \neq S(M_j)$ and $M_i \mathcal{K}_\pi M_j$. If π is non-injective on evidence atoms, there exist contracts with such a collision witness.*

PROOF. The first statement unfolds the definitions: $\mathcal{K}_\pi$ is equality after projection, while exact non-equivalence is $S(M_i) \neq S(M_j)$. For the non-injective case, take distinct evidence atoms $(a_1, s_1) \neq (a_2, s_2)$ with $\pi(a_1, s_1) = \pi(a_2, s_2)$. Construct M_1 so that $S(M_1) = \{(a_1, s_1)\}$ and M_2 so that $S(M_2) = \{(a_2, s_2)\}$, with all other actions UNSPEC. These minimal contracts are well formed because action states are assigned independently. Then $S(M_1) \neq S(M_2)$, but $S_\pi(M_1) = S_\pi(M_2)$ is the same singleton, so (M_1, M_2) is a collision witness. $\square$

DEFINITION 4 (PROJECTION INFORMATION PROFILE). *For a finite set of observed maps Y , let $H(S)$ be the empirical Shannon entropy of reference signatures and $H(S_\pi)$ the entropy of projected signatures. The entropy-retention ratio $\rho_\pi = H(S_\pi)/H(S)$ measures distinguishability retained by the comparison vocabulary. The projected collision count is the number of projected-signature classes containing more than one reference signature class.*

LEMMA 3 (KERNEL INFLATION). *If a projection declares E_π equivalent pairs and reference comparison declares E over the same pair set, then E_π/E is the pairwise inflation factor induced by the kernel. Every collision witness contributes to this inflation, and a zero-collision strengthened projection must have $E_\pi = E$.*

PROOF. Reference equivalence implies projected equivalence only when the projection is a function of the full atom payload, so $E_\pi \geq E$. The additional pairs counted by E_π are exactly the unequal reference signatures that collapse under π , which are collision witnesses. If there are none, no additional pairs are counted and the two denominators agree. $\square$

OBSERVATION (EVIDENCE REFINEMENT). *Adding a non-conflicting evidence-backed rule cannot silently weaken a public contract: it behaviorally refines possible worlds, evidentially refines support, or both.*

The remaining notation is the payoff of the model: it turns an informal vocabulary dispute into a finite certificate check. Let F be the semantic-field universe and let π_R append field values in $R \subseteq F$ to the baseline projection. For a finite comparison set X , define $X_{\pi_R} = \{(M_i, M_j) \in X \mid S(M_i) \neq S(M_j) \wedge M_i \not\mathcal{K}_{\pi_R} M_j\}$. A retained-field set R is safe for π on X when $X_{\pi_R} = \emptyset$; otherwise any pair in X_{π_R} is a counter-witness. Since adding retained fields can split projected classes but cannot merge already separated classes, safe sets are upward closed and unsafe sets are downward closed. For each witness w , the separator family $\mathcal{D}_w = \{R \subseteq F \mid M_i \not\mathcal{K}_{\pi_R} M_j\}$ is upward closed, so a universal retained-field separator is the finite intersection problem $R \in \bigcap_{w \in X_\pi} \mathcal{D}_w$. The reported certificates therefore consist of all separated witnesses plus counter-witnesses for smaller field sets; exhaustive enumeration over $2^{|F|}$ retained-field subsets is complete for the declared field universe.

PROPOSITION 1 (MINIMUM RETAINED-FIELD SEPARATOR AS HITTING SET). *For finite semantic fields F and collision witnesses X_π , deciding whether there is a universal retained-field separator of size at most k is NP-complete in $|F| + |X_\pi|$. For OptSem-C’s fixed field universe, minimum separators are exactly enumerable.*

PROOF. Membership is finite checking. Hardness reduces HITTING SET: create one witness per set $H_i \subseteq F$ whose two contracts

differ exactly on H_i ; a retained-field set separates that witness iff it intersects H_i . $\square$

The semantics makes the comparison payload explicit. A user comparing only operator names chooses a projection that drops placement, decision time, variant, and observability. A feature checklist chooses an even coarser projection that can drop operator class itself. OptSem-C does not forbid those projections; it computes exactly which kernel collisions they induce, which grounded counter-witnesses remain, and which query-processing fields separate the comparison within the declared corpus.

4 CONTRACT PROBE SUITE AND GROUNDED CORPUS

OptSemBench-C is a contract-probe suite: it generates executable SQL representatives that cover optimizer-relevant feature interactions and measures which public contracts those probes activate. It deliberately does not rank engines by latency. This choice follows the measurement principle that the metric and denominator must match the object of study: large-scale system comparisons measure analytical designs [55], while OptSemBench-C measures whether the comparison vocabulary preserves public optimizer distinctions. Cardinality benchmarks ask whether estimates improve optimizer decisions, as in CardEst [36] and CardBench [17]. Learned estimators such as Learned Cardinalities [43], NeuroCard [68], DeepDB [37], and FactorJoin [67] model distributions or joins; learned optimizers such as Neo [48] and Bao [47] test plan selection or hinting. OptSemBench-C asks a prior question: before measuring plan quality, did the comparison vocabulary preserve the public optimizer distinctions being compared?

The contract schema is fixed before source extraction, witness counting, and retained-field search. Its fields come from prior optimizer architecture rather than from the measured collisions. Operator/action and layer follow System R [60], Volcano [30], Cascades [31], Calcite [8], and Orca [62]. Placement follows Garlic federation [12], BigDAWG polystores [27], Dremel serving trees [50], and Snowflake cloud execution [23]. Decision time follows progressive optimization [49], mid-query reoptimization [41], adaptive query processing [26], eddies [7], and Spark adaptive execution [6]. Observability is admitted from plan, profile, and stage evidence in the public source ledger, with provenance-style evidence separation [34]. Layer and placement enter the field universe as architecture-derived dimensions; exhaustive retained-field testing later reports them as an interpretable sufficient basis among finite certificates, not as a universal separator learned from private behavior.

Let $F = F_1 \times \dots \times F_n$ be the finite feature domain and Ψ the validity constraints. A target interaction is a valid partial assignment over a feature subset. The schema and feature domain are fixed before witness counting and derive from standard query-processing constructs and cited public optimizer surfaces, not from witnesses or motifs. The probe suite targets global pairwise coverage plus selected three-way interactions for source boundary, connector capability, operator class, statistics need, join shape, adaptivity trigger, and distribution trigger. The generator builds one deterministic candidate pool from renderable feature interactions and admitted public-rule guards. It first includes 99 guard-forced completions so

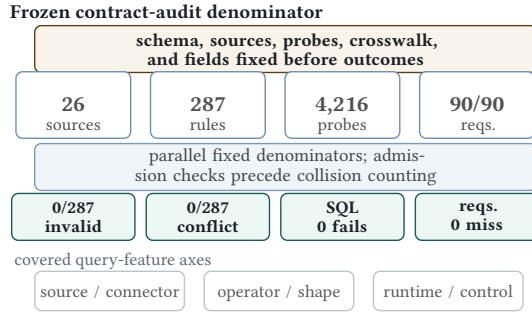


Figure 3: Corpus, probe-suite, and execution denominators: 26 public sources, 287 admitted rules, 4,216 executable probes, and 90/90 published optimizer-decision requirements represented by 71 distinct generated SQL probes. These are parallel fixed denominators; zero-failure checks validate support, consistency, SQL execution, and feature-space representation before collision measurement.

that every admitted rule has a reachable query shape, then greedily covers the remaining feature-interaction universe. These two roles are reported separately: guard-forced probes prevent public contracts from becoming unreachable paper claims, while the remaining probes complete the declared feature-interaction denominator. Neither role is treated as an independent held-out population sample. A probe is useful when it activates a contract guard or separates two contract signatures, not when it estimates workload frequency.

OBSERVATION (COVERAGE). *If every valid target interaction has at least one renderable full assignment, the generator terminates with a probe set covering every renderable target interaction.*

PROOF. The target universe is finite. Each selected assignment covers at least one uncovered interaction. Thus the number of uncovered interactions decreases monotonically until it reaches zero. $\square$

The main empirical corpus contains only 287 source-linked grounded rules. Each rule records a public evidence span, source URL, line range, digest, and source class. Claims without grounded evidence, expressible guards, or public observability are outside the finite denominator rather than negative evidence. Thus the paper claims coverage of the declared public-contract denominator, not a census of private optimizer rules. Figure 3 gives the denominator view before any collision measurement: sources, rules, probes, and feature-space requirements are fixed, and the zero-failure checks verify support, consistency, SQL execution, and crosswalk coverage. The corpus covers plan observability, pushdown, join search, distribution, materialization, runtime adaptivity, statistics, and optimizer control surfaces. Every grounded rule is triggered by at least one generated probe. Zero admitted merge conflicts means unresolved MUST/MUST_NOT evidence has been split by guard/version or excluded before admission, not that public documentation never conflicts.

The source set is fixed before witness counting. We admit an engine only when public sources provide: a comparable plan or

contract surface, enough detail to extract an action and applicability guard, and stable analytical-SQL behavior rather than a private vendor note. PostgreSQL, Trino, BigQuery, Spark SQL, Snowflake, DuckDB, and ClickHouse enter through versioned source records in the archived evidence ledger named by the page-1 availability DOI and SHA; Calcite and DataFusion source records provide schema context rather than denominator engines. These records are not scholarly support and not performance measurements. They are the measured public contracts that the framework formalizes. Proprietary workload logs, undocumented cost-model traces, future engine versions, and operator families outside the declared feature domain are excluded rather than silently counted. This admission rule makes documentation density a visible confound: source-rich engines can create more measured collision opportunities, so the paper reports finite source sensitivity rather than representativeness.

The source-linked denominator spans seven engines: PostgreSQL contributes 80 admitted rules, Trino 61, BigQuery 37, Spark SQL 36, ClickHouse and Snowflake 26 each, and DuckDB 21; the largest single source supports 46 rules. This design separates admission, probing, and execution validation. Public sources admit rules; generated probes test guard reachability and SQL executability; DuckDB and PostgreSQL runs validate the probe denominator on two open engines, not every vendor contract. All-engine execution would conflate SQL validity with public-contract admission. A DuckDB source span can admit a DuckDB public contract, but a later DuckDB execution is only SQL execution and visible-plan sanity evidence; it never admits rules or chooses a separating field. Conversely, engines without local execution still participate in collision measurements only through source-linked public contracts, source leave-one-out checks, and side-balanced witness support.

The grounded corpus is intentionally conservative. A rule is admitted only when the evidence span supports an optimizer action, a guard or applicability condition, and an observability or placement interpretation. Vague performance advice is excluded from the main corpus. This policy reduces corpus size but strengthens the central claim: the evaluation is about contracts that are publicly supported, not about inferred implementation behavior.

Contract extraction and validation checks. Each admitted rule records the engine, version scope, source identifier, digest, action fields, guard, and observability surface. The admission codebook is operational rather than statistical: a span must name an optimizer action, state an applicability condition or guard, expose observability or execution placement, and map to a versioned public locator; otherwise it is excluded from the denominator rather than converted into negative evidence. Ambiguous, advisory-only, or performance-only prose is excluded by the same rule. Public contracts are therefore constructed interpretations of public evidence, not objective claims about hidden optimizer behavior. Automated checks validate schema consistency, source linkage, guard reachability, trigger coverage, merge conflicts, SQL executability, and table-source consistency. This is a single-extractor source-line protocol rather than formal verification or recoding agreement, so the paper does not claim inter-rater reliability. The reliability claim is instead auditability: every accepted row has a locator, line span, digest, guard, paraphrase, and regenerated downstream witness; every excluded row is outside the declared denominator rather

Table 3: Source-to-rule traceability for representative contracts. Each row binds a rule state/action to a public source span, digest prefix, and paraphrase used by the replay manifest.

Contract rule			Auditable evidence	
Engine	State	Action	Span/digest	Source-supported paraphrase
PostgreSQL	MAY	Plan/observe/local	PG-EXPLAIN-001 bfb47e66 L46-L56	EXPLAIN exposes estimated startup and total cost, output rows, row width, and planner-cost units.
BigQuery	MAY	Exchange/adapt/cloud	BIGQUERY-PLAN-001 cc7b206a L990-L992	The query plan may be modified during execution by adding repartition stages.
Snowflake	MAY	Plan/observe/cloud	SNOWFLAKE-EXPLAIN-001 93a61359 L150-L154	EXPLAIN compiles a statement and returns a logical execution plan rather than executing it.
ClickHouse	MAY	Join/adapt/local	CLICKHOUSE-JOINING-001 a1f757e2 L313	With automatic join-algorithm selection, execution can switch from hash join to partial merge join.
DuckDB	MAY	Materialize/inline/local	DUCK-WITH-001 2a60c8ab L482-L487	CTE inlining is guarded by single use, absence of volatility, no grouped aggregation, and inlining hints.
Spark SQL	MAY	Plan/adapt/distributed	SPARK-PERF-001 7e5e2b00 L129-L133	Adaptive query execution uses runtime statistics and can re-optimize a query while it is running.
Trino	MUST_NOT	Agg./pushdown/source	TRINO-PUSH-001 cc177219 L168-L176	Aggregation pushdown does not support aggregate calls containing expressions.

than hidden as a negative. The public-provenance ledger contains 287 verified rules, 287 source segments, 26 source digests, retrieval dates, paraphrase policy, and zero public-provenance issues under replay checks; the table-source audit maps all paper tables to regenerated evidence files. Table 3 gives representative spans; Table 8 exposes the source-surface catalog; the replay materials carry the complete rule-to-source manifest, source digests, claim-evidence graph, regenerated table manifests, per-row surface flags, and a recoding worksheet with blank independent-coding columns for independent follow-up. This evidence chain lets an auditor recode or dispute any row without trusting hidden notes, but it remains a manual public-contract extraction study whose corpus composition affects the result.

The field schema and projection portfolio are replay inputs fixed before collision counting. The portfolio is chosen because the admitted public surfaces expose keyword/operator labels and a smaller yes/no control family; it does not exhaust all possible optimizer vocabularies. Layer plus placement is reported as an interpretable separating basis for headline witnesses after enumerating retained-field vocabularies; the frontier tables give scoped safety and do not claim global safety. Public source records admit claims; EXPLAIN, profile, or stage surfaces bind exposed plan evidence when available; SQL execution validates SQL constructability and feature representation rather than private optimizer truth. A counted witness requires two source-supported reference signatures, projection equality under the declared vocabulary, and separation by the reported field.

The denominator dashboard in Table 4 checks that this conservatism does not create unreachable contracts or syntactic probes. A finite-disjunction guard semantics evaluates each admitted rule against the generated probe denominator. All 287 grounded rules have at least one triggering probe; 252 depend on query features rather than global engine labels; and no guard uses a feature dimension or value outside the probe-suite domain.

Coverage is reported over two denominators. The first is probe-suite coverage: every targeted renderable valid feature interaction is covered by at least one generated probe. The second is contract triggering: every grounded rule is applicable to at least one probe. Together these denominators make the empirical scope explicit. A

rule that cannot be triggered by any probe would indicate a probe-suite gap; a feature interaction that cannot be rendered, lacks a source-supported optimizer action, or falls outside the declared feature domain is excluded from the coverage denominator and reported as a modeling constraint. Thus “90/90” below means coverage of declared optimizer-decision requirements inside the feature schema, not coverage of all behavior present in JOB, TPC-H, TPC-DS, or any vendor engine.

The benchmark also exposes executable SQL shapes. Each generated probe has a normalized SQL skeleton and shape checks for joins, filters, grouping, ordering, limits, and reuse structure. The full 4,216-probe corpus is accompanied by a compact 48-probe motif-cover prefix. The real-engine motif execution set uses the deterministic union of this motif cover and residual feature-axis maxima; it is not a hand-picked success set, and it is not a replacement for the full evaluation.

A common failure mode in generated probe suites is to produce feature-complete but non-executable query text. OptSemBench-C therefore validates every SQL skeleton on a deterministic relational catalog with local, remote, and cross-source namespaces. This validation is not a performance baseline, contract-truth check, or exhaustive trigger proof. Its role is stricter and more basic: every generated probe must be parseable, plannable, executable, and shape-consistent with its feature vector. Table 4 reports the validation result: all 4,216 generated probes obtain a plan and execute, with zero SQL-shape failures. Repeating the full corpus across three deterministic catalog populations yields 12,648 successful probe-catalog executions, and DuckDB/PostgreSQL checks add 8,432 full-corpus engine-probe executions plus 142 motif-representative executions with zero execution failures.

Contract validation is layered. Guard reachability says that each public rule has a generated probe satisfying its admitted guard; SQL execution says that the denominator is not syntactic fiction. Visible-plan sanity checks on DuckDB and PostgreSQL expose expected plan tokens at mean rates 0.705 and 0.796 over the full corpus, and 0.756 and 0.819 on feature-space representatives. The missing 20–30% is a reported observability limit: joins match completely, while aggregate and CTE labels are often absent from exposed plan text. These rows bound generated-SQL quality and observability; they do

Table 4: Probe-suite denominator and validation checks across corpus, SQL, open-engine execution, and feature-space crosswalk layers. Bold marks complete coverage or zero SQL-shape failures; execution rows validate constructability, not contract truth.

Layer	Check	Value	Scope	Interpretation
Corpus	grounded rules	287	26 sources	finite public-contract relation
Projection	public surfaces	26	26 kw/op; 12 yes/no	catalog, not practice survey
Corpus	probe-supported rules	287/287	median support 102	every admitted rule is triggerable
Corpus	non-global guards	252	3 containments	query-conditional, audited overlaps
Guard schema	invalid dimensions	0	fixed feature domain	no out-of-domain guard values
SQL corpus	generated probes	4,216	digest df107f018e37	full executable denominator
SQL corpus	feature-cover prefix	48	human inspection	compact replay slice, not cherry-picking
SQL execution	planned/executed probes	4,216/4,216	91 distinct plans	every generated probe plans and runs
SQL execution	shape/exec failures	0/0	18,448 rows	no SQL-shape or runtime failures
Real engines	full DuckDB/PostgreSQL	8,432/8,432	tags .705/.796	SQL constructability, not source truth
Real engines	feature reps DuckDB/PostgreSQL	142/142	71 reps $\times$ 2	budgeted open-engine sanity
Contract truth	private optimizer behavior	–	closed engines	not inferred from execution
Crosswalk	feature requirements	90/90	71 SQL reps	not workload substitution

Table 5: Probe budgets for triggering grounded contract rules; random columns summarize subset baselines. Bold marks complete trigger coverage.

Budget	Cover	Diagnostic	Random mean	Random 5–95
50	.672	.997	.707	[.627,.767]
100	1.000	1.000	.811	[.770,.850]
250	1.000	1.000	.890	[.864,.913]
500	1.000	1.000	.920	[.899,.941]
1000	1.000	1.000	.951	[.934,.969]
2000	1.000	1.000	.973	[.958,.986]
4216	1.000	1.000	1.000	[1.000,1.000]

not validate closed-source contracts, reverse-engineer private optimizers, prove every documented behavior fires on every cost-model instance, or cross-validate a source by running the same engine. A counted witness still requires source-supported reference signatures, guard reachability, projection equality, and field separation; runtime plan satisfaction is a post-admission SQL-denominator validation layer and is excluded by a non-interference check from collision tables, while executability constraints still delimit which feature interactions enter the probe suite.

The same fixed probes support two budgeted evaluation orders. The cover order is the generator’s feature-interaction order. The diagnostic order greedily maximizes newly triggered grounded rules without adding any new probe; it is a rule-coverage schedule over admitted contracts, not an unseen workload sample. Table 5 reports both. The cover order triggers all grounded rules within 100 probes; the diagnostic order triggers 286 of 287 rules within 50 probes and all rules within 100. Random subsets of 100 probes trigger 81% of rules on average and 85% at the 95th percentile. Thus OptSemBench-C is not merely large enough to cover the surface; it also contains compact diagnostic prefixes that expose almost all grounded contracts.

A probe suite for public optimizer contracts must connect to existing database workloads without becoming another latency benchmark. We therefore crosswalk published optimizer and benchmark motifs to OptSemBench-C feature requirements: JOB join-order sensitivity [45], POLAR-style join structure [40], TPC decision-support benchmarking [56], TPC-DS design [52], SSB [54], LDBC [28], Wisconsin [9], warehouse surveys [24], YCSB [22], and LakeBench [25].

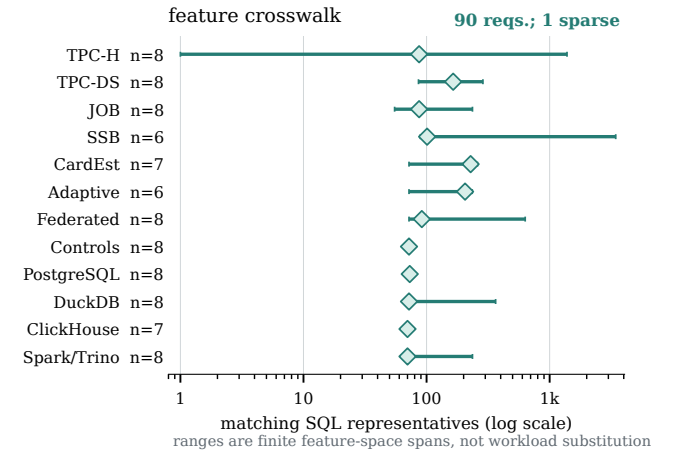


Figure 4: Feature-space crosswalk from the frozen table of 12 published motif families and 90 optimizer-decision requirements. These requirements validate through 71 distinct SQL representatives; a matching-probe span is the minimum-maximum number of OptSem-C probes that trigger a family, so sparse requirements stay visible. The figure is a feature-space denominator check, not a claim that generated probes reproduce JOB, TPC-H, or TPC-DS workloads.

Figure 4 reports a feature-space crosswalk, not workload substitution: 90 declared optimizer-decision requirements across 12 published families are represented by 71 distinct generated SQL probes because several probes satisfy more than one requirement. Five anchors illustrate the span: JOB join-order search (8/8 requirements), exact-cardinality testing (9/9), CardEst-style plan-quality sensitivity (8/8), adaptive replanning (9/9), and cross-source joinability (7/7). Table 6 gives the 12-family matching-probe distribution. The selected probes are feature-interaction representatives, not replacements for JOB or TPC-H queries, and the check is schema-adjacent rather than independent workload validation. A counted collision still requires two source-supported signatures, projection equality, side-balanced support, and field separation. We claim contract-denominator adequacy and feature-space representation, not workload reproduction.

Table 6: Matching-probe difficulty across published optimizer feature requirements. Underlining marks the sparsest and broadest feature-space coverage extremes; requirement-family sources are cited in the crosswalk paragraph.

Suite	Reqs.	Min	Median	Max
TPC-H	8	<u>1</u>	87.0	1384
TPC-DS	8	86	164.5	287
JOB	8	55	87.0	236
SSB	6	89	101.0	<u>3442</u>
CardEst	7	72	228.0	259
Adaptive	6	72	205.5	235
Federated	8	72	91.5	633
Controls	8	70	72.0	73
PostgreSQL	8	72	73.0	73
DuckDB	8	68	72.0	364
ClickHouse	7	70	70.0	70
Spark/Trino	8	69	70.0	236

The feature-space difficulty view prevents a misleading denominator. Some requirements are broad and match many probes, such as star-schema group-by shapes. Others are sparse, such as a TPC-H conjunctive-filter requirement that has one matching rendered probe under the current validity constraints. Reporting both coverage and matching-probe counts makes probe-suite adequacy inspectable: coverage says the requirement is present, while difficulty says how concentrated or redundant that coverage is. The executable crosswalk adds a second denominator check: for each published requirement, the system selects one satisfying probe and validates the generated SQL on the deterministic catalog, yielding 90 validated requirements, 71 distinct representatives, and 383 result rows. Thus the denominator is not a list of names; it is a set of optimizer-decision requirements with concrete SQL representatives and an explicit many-to-one map. Engine-plan validation is a separate experiment because the crosswalk is a denominator check rather than a claim that published benchmark queries and generated probes induce identical plans.

5 EVALUATION

The evaluation follows the objection path a comparison paper would face. Do coarse public vocabularies create denominator errors? Are counts grounded by witnesses rather than names? Do source, probe, feature, and engine stresses preserve a finite kernel without becoming population claims? Are projection-audit stages cheap enough to rerun when the denominator changes? The schema, projections, source manifest, source-admission codebook, and retained-field universe are frozen before counting; exhaustive retained-field search is reported as a certificate check, while layer and placement are architecture-derived dimensions. Execution is a post-freeze SQL-denominator consistency check: it validates probes but never admits rules or chooses fields; a non-interference check verifies that DuckDB/PostgreSQL validation files are not consumed by collision tables. The table column “Coll.” is contract-level: a declared vocabulary collapses unequal public optimizer-contract signatures before any latency, plan-quality, or cost claim is interpreted.

The comparison surfaces are executable projection vocabularies and controls, not DBMS bug finders such as SQLancer [59], NoREC [58], or other SQL fuzzing tools. Those tools test execution

Table 7: Projection-collision summary and separating fields inside the frozen corpus.

Projection	Eq.	Coll.↓	Corpus Coll./Eq.↓	Δ mean/max	Diagnosis
Exact	2,030	0	0.0%	0.00/0	ctrl
Keyword*	2,284	254	11.1%	9.09/17	placement
Yes/No	2,036	6	0.3%	3.00/3	layer
Operator-only*	2,268	238	10.5%	5.84/14	layer
Placement-only	12,732	10,702	84.1%	21.29/53	stress
State-only	25,623	<u>23,593</u>	<u>92.1%</u>	<u>13.06/64</u>	stress
Surface	2,030	0	0.0%	0.00/0	sep.

Note. Coll./Eq. is the fraction of declared equivalences that collapse unequal reference signatures in this corpus, not a population rate. Δ counts differing public atom fields. Stars mark headline public vocabularies; bold zeros mark controls or separating-field certificates.

behavior under generated oracles; OptSem-C audits whether a public comparison vocabulary has already collapsed unequal contracts. Across the 26 public source surfaces, all expose keyword-style or operator-family labels, twelve expose yes/no-style controls, and none exposes the complete reference contract payload. Reference comparison is the negative control, one-field projections are diagnostic ablations, and strengthened projections restore fields to localize missing information. All projections are fixed before witness counting and evaluated over the same source-linked maps. The question is how many witnesses are lost if equality is decided at that vocabulary, not whether every cited study made that exact projection.

Coarse comparison induces concentrated, separable collisions. The main empirical result is a finite counterexample to two coarse optimizer-comparison vocabularies, plus a positive result about narrower controls. Table 7 keeps public vocabularies, controls, stress rows, and separating fields on the same scale: keyword declares 2,284 equivalences and collapses 254 unequal reference signatures, while operator-only declares 2,268 and collapses 238. These 11.1% and 10.5% fractions condition on equivalences declared by the tested vocabularies inside this corpus, not on optimizer papers. The 492 primary records are concentrated: keyword records are BigQuery–ClickHouse (203), BigQuery–Trino (45), and ClickHouse–Trino (6); operator-only records are ClickHouse–Trino (123) and Spark SQL–Trino (115). The remaining 17 engine pairs have zero headline collisions, so coarse labels often suffice for local-engine or non-boundary comparisons in this denominator. The risk signal appears at cloud service, connector, distributed execution, and runtime-adaptation boundaries where public layer and placement distinctions carry the comparison. The yes/no public-control view declares 2,036 equivalences but collapses only six, all in ClickHouse–Trino, a narrow positive result for enablement-style controls. Primary collision pairs have reference-signature Jaccard distance 1.0 and differ by 9.09 and 5.84 evidence atoms on average. The strict reference and strengthened surface controls have zero collapsed pairs; one-field stress rows show that a single optimizer dimension can make the denominator worse rather than safer.

Representative witness audits appeared earlier in Table 1; the evaluation now asks whether those examples are isolated or part of a finite kernel.

Table 8 brings the 26-row public-surface catalog into the paper. It reports source documents that admit comparison vocabularies,

Table 8: Paper-facing public-surface catalog. K means keyword or feature-name labels, Y/N means enablement-style controls, Op means operator-family labels, and Ref means the full OptSem-C reference-signature payload. Counts are source documents, not engine prevalence; Rule% exposes documentation density in the frozen denominator.

Engine	Src.	Rules	Rule%	K	Y/N	Op	Ref
BigQuery	4	37	12.9	4	0	4	0
ClickHouse	4	26	9.1	4	1	4	0
DuckDB	1	21	7.3	1	1	1	0
PostgreSQL	4	80	27.9	4	2	4	0
Snowflake	3	26	9.1	3	0	3	0
Spark SQL	4	36	12.5	4	3	4	0
Trino	6	61	21.3	6	5	6	0
Total	26	287	100.0	26	12	26	0

not how often researchers use them. Every admitted source exposes keyword or operator-family labels, twelve expose yes/no controls, and none exposes the full reference-signature payload. PostgreSQL and Trino contribute 27.9% and 21.3% of admitted rules, while DuckDB and Snowflake contribute 7.3% and 9.1%; source-rich engines can create more measured opportunities, so pair counts are corpus-specific evidence rather than documentation-normalized prevalence. If a comparison falls outside this finite catalog, the safe action is to rerun extraction rather than transfer the certificate.

Finite resampling keeps keyword at roughly 10–12% and operator labels at roughly 9–12% of corpus-specific declared equivalences. Single-source removal is stricter to read: removing one source can delete original witnesses and create new ones, so the audit reports both nonzero counts and witness-identity movement rather than treating leave-one-source as population sampling. The diagnosis column localizes the missing semantics: placement separates keyword, layer separates the operator row, and the strengthened semantic surface returns to zero collapsed pairs. The sparse yes/no row is a positive boundary result for enablement-style controls: its six witnesses are all ClickHouse–Trino records and it is not used to claim broad binary-control risk.

Projection kernels are measurable objects. Figure 5 reports the finite information profile. Reference signatures form 1,288 observed classes. Keyword projection compresses them into 148 projected classes, retains 64% of empirical signature entropy, and inflates declared equivalence by 12.5%; operator-only retains more entropy but still leaves 238 pairwise collisions. Placement-only and state-only surfaces show the opposite failure mode: a single field creates very large kernels. The strengthened surface still has projected collisions at the signature-class level, but no pairwise collision over the engine-probe comparison set.

Resolution lattice stress test. Table 9 enumerates all 256 retained-field subsets over the same 88,536 engine–probe comparisons. The empty vocabulary declares 59,546 unsupported equivalences, and every singleton semantic field remains unsafe: operator leaves 238 collisions, kind 252, layer 449, and placement 10,702. Excluding action-variant identifiers, the semantic lattice has 128 subsets, 44 unsafe regions, minimum safe field count two, 14 minimal safe vocabularies, and eight maximal unsafe vocabularies. Ten

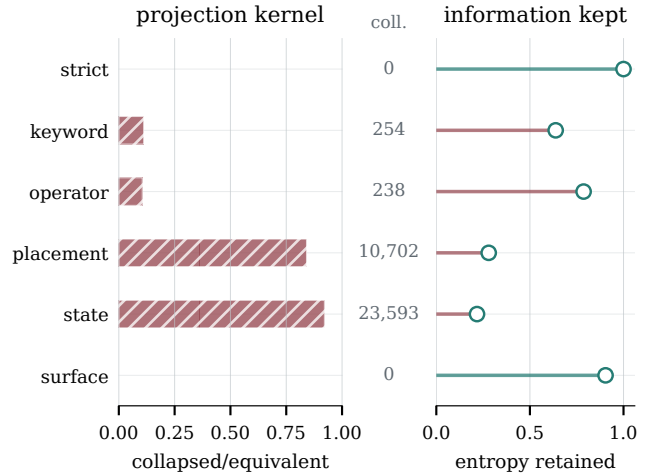


Figure 5: Projection information profile. Left: collapsed unequal contracts among declared equivalent pairs. Right: empirical signature entropy retained by each projection vocabulary.

Table 9: Retained-field lattice for projection separation. Ent. kept is higher-better, Eq. is denominator size, and Coll. is lower-better. All sem. excludes action-variant identifiers, so entropy can remain below 1.0. Bold zeros mark safe vocabularies; underlining marks the largest unsafe collision count.

Fields	R	Classes	Ent. kept↑	Eq. Coll.↓
None	0	1	0.000	61,576 <u>59,546</u>
Operator	1	387	0.787	2,268 <u>238</u>
Action kind	1	183	0.661	2,282 <u>252</u>
Layer	1	52	0.552	2,479 <u>449</u>
Placement	1	9	0.280	12,732 <u>10,702</u>
Operator+layer	2	591	0.850	2,030 0
Kind+placement	2	214	0.680	2,030 0
Surface	5	820	0.904	2,030 0
All sem.	7	947	0.934	2,030 0

concrete engine–probe witnesses cover all unsafe retained-field regions.

The robustness view in Table 10 checks that the headline result is not an effect of a single denominator. The micro fraction conditions on declared equivalence; finite bands and probe-cluster resampling summarize dispersion without population inference; leave-one-source removes all rules supported by one public source. Keyword and operator-only remain nonzero, with leave-one-source conditional-fraction ranges [0.017, 0.144] and [0.032, 0.148]; the Boundary panel records retained and newly induced witnesses.

The evidence side is cross-checked without retraining on headline records: every collision has at least two public source records and side-balanced support, and the freeze manifest binds schema, sources, projections, feature-space crosswalk, implementation, and replay entrypoints before witness counting.

The Boundary panel in Table 10 is a ledger rather than a victory table. Keyword and operator-only remain nonzero after every leave-one-source deletion, but witness identities move: keyword

Table 10: Freeze, robustness, boundary, and replay dashboard. Bold zeros mark controls or separating-field surfaces; collapsed-comparison fractions are finite-corpus summaries, and source leave-one-out is an information-deletion check rather than a sampling estimate.

Panel	Row	Coll.↓	Frac./Cost	Stress	Boundary read
Freeze	Denominator	–	pre-count	SHA manifest	schema, sources, projections, replay locked
Collision	Strict reference	0	0.000	control	no collapsed public signatures
Collision	Keyword labels	254	0.111	finite [.099,.125]	keywords hide placement
Collision	Operator labels	238	0.105	finite [.093,.118]	operators hide optimizer layer
Collision	Strengthened surface	0	0.000	separated	retained fields remove witnesses
Boundary	Source LOO	–	nonzero 26/26	kept/new tracked	deletion check, not sampling
Boundary	Pair + basis	0	492+6 separated	4 pairs; 486 probes	scoped layer+placement basis
Replay	Replay total	–	3.50 s	455,828 input rows	excludes admission/package build
Replay	8× audit lift	–	2.49 s	708,288 checks/projection	inner-loop lift, not new engines

retains as little as 2% of original witnesses in the most disruptive deletion and can induce new witnesses. Thus robustness is not witness-identity stability, and source removal is information deletion rather than a new engine-population sample. The yes/no projection is source-sensitive and stays out of the broad robustness claim; layer+placement resolves feature-family and engine-family stress, while point-learned engine-pair bases fail on held-out pairs.

Because the 287-rule corpus is finite, the remaining checks test whether the kernel is a one-source or one-shape accident. The 492 primary collapsed comparison records plus six yes/no boundary records occupy four observed engine pairs and 486 distinct probes, touch all 12 feature dimensions, and fully cover the value domain for 10 of them; every probe carries at most one counted witness.

Finite-audit replay cost. The Replay panel in Table 10 reports deterministic replay bounds, not stochastic performance comparisons: full audits cover 88,536 engine–probe checks per projection above 50,000 checks/s; the 8× lift repeats the relation to 708,288 checks per projection above 190,000 checks/s with prefix-scaling $R^2 \geq 0.98$; streaming maintenance has zero drift.

Field certificates test an architecture hypothesis. A certificate must match the tested projection, differ under reference contracts, separate under the proposed fields, and survive smaller-field counter-witnesses. Placement separates keyword collisions, layer separates operator-only collisions, and layer+placement separates the 492 primary records plus the six yes/no boundary records without action-variant labels. Other retained-field subsets can be safe for narrower projections; we report layer+placement as the uniform, architecture-readable basis that covers the headline keyword, operator-only, and yes/no families in this corpus. This is a finite certificate, not a claim that other papers must use exactly these fields. The hypothesis is falsifiable because operator leaves six keyword collisions, placement alone fails operator, and point-learned engine-pair bases fail held-out pairs. Figure 6 gives the scoped frontier; layer+placement is a counted-witness basis, not a globally safe vocabulary.

core state-free separation frontier

U=unsafe S=first safe -=already safe					
retained depth					
	0	1	2+	basis	safe/unsafe
keyword (254)	U	S	–	P (4)	23/9
yes/no (6)	U	S	–	P (2)	24/8
operator (238)	U	S	–	L (3)	22/10
all (498)	U	U	S	L+P (6)	21/11

Figure 6: Scoped separation frontier. U means an unsafe retained-field depth with at least one collapsed comparison, S marks the first safe depth, dash means already safe, and parenthesized values count minimum separating bases.

6 LIMITATIONS AND CONCLUSION

OptSem-C is deliberately finite. It audits a source-linked public-contract denominator, not private optimizer behavior, workload prevalence, or a population rate for database papers. The current evidence chain is a single-extractor source-line protocol with replayable locators, digests, guards, paraphrases, downstream witnesses, and a recoding worksheet for independent follow-up; it does not claim inter-rater reliability. DuckDB/PostgreSQL runs validate SQL constructability and visible-plan sanity for the generated denominator, not the truth of closed-source contracts.

Within that scope, the counted mechanisms are clear. Placement separates local, delegated, distributed, and cloud execution; layer and decision time separate rewrite, planning, distribution, and runtime adaptation. Across the 498 counted records, the ledger attributes differences to action variants (498), placement (466), layer (430), plan evidence (366), decision time (362), and operator family (281). The practical workflow is short: name the coarse vocabulary, extract source-supported contracts, project them to the comparison table, and either retain the separating fields or state the intentionally coarse scope. This is narrow but useful before latency or portability claims at federated SQL boundaries.

REFERENCES

- [1] Daniel Abadi, Peter Boncz, Stavros Harizopoulos, Stratos Idreos, and Samuel Madden. 2013. The Design and Implementation of Modern Column-Oriented Database Systems. *Foundations and Trends in Databases* 5, 3 (2013), 197–280. <https://doi.org/10.1561/19000000024>
- [2] Daniel J. Abadi, Samuel R. Madden, and Miguel Ferreira. 2006. Integrating Compression and Execution in Column-Oriented Database Systems. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 671–682. <https://doi.org/10.1145/1142473.1142548>
- [3] Daniel J. Abadi, Samuel R. Madden, and Nabil Hachem. 2008. Column-Stores vs. Row-Stores: How Different Are They Really?. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 967–980. <https://doi.org/10.1145/1376616.1376712>
- [4] Daniel J. Abadi, Daniel S. Myers, David J. DeWitt, and Samuel R. Madden. 2007. Materialization Strategies in a Column-Oriented DBMS. In *Proceedings of the IEEE International Conference on Data Engineering*. 466–475. <https://www.cs.umd.edu/~abadi/papers/abadiicde2007.pdf>
- [5] A. V. Aho, Y. Sagiv, and J. D. Ullman. 1979. Equivalences among Relational Expressions. *SIAM J. Comput.* 8, 2 (1979), 218–246. <https://doi.org/10.1137/0208017>
- [6] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1383–1394. <https://doi.org/10.1145/2723372.2742797>
- [7] R. Avnur and J. M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 261–272. <https://doi.org/10.1145/342009.335420>
- [8] E. Begoli, J. Camacho-Rodriguez, J. Hyde, M. J. Mior, and D. Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 221–230. <https://doi.org/10.1145/3183713.3190662>
- [9] D. Bitton, D. J. DeWitt, and C. Turbyfill. 1983. Benchmarking Database Systems: A Systematic Approach. In *Proceedings of the International Conference on Very Large Data Bases*. 8–19. <https://www.vldb.org/conf/1983/P008.PDF>
- [10] P. A. Boncz, M. Zukowski, and N. Nes. 2005. MonetDB/X100: Hyper-Pipelining Query Execution. In *Proceedings of the Conference on Innovative Data Systems Research*. 225–237. <https://www.cidrdb.org/cidr2005/papers/P19.pdf>
- [11] P. Buneman, S. Khanna, and W.-C. Tan. 2001. Why and Where: A Characterization of Data Provenance. In *Proceedings of the International Conference on Database Theory*. 316–330. https://doi.org/10.1007/3-540-44503-X_20
- [12] Michael J. Carey, Laura M. Haas, Peter M. Schwarz, Manish Arya, William F. Cody, Ronald Fagin, Myron Flickner, Allen W. Luniewski, Wayne Niblack, Dragutin Petkovic, John Thomas, John H. Williams, and Edward L. Wimmers. 1995. Towards Heterogeneous Multimedia Information Systems: The Garlic Approach. In *Proceedings of the International Workshop on Research Issues in Data Engineering*. 124–131. <https://doi.org/10.1109/RIDE.1995.378736>
- [13] A. K. Chandra and P. M. Merlin. 1977. Optimal Implementation of Conjunctive Queries in Relational Data Bases. In *Proceedings of the ACM Symposium on Theory of Computing*. 77–90. <https://doi.org/10.1145/800105.803397>
- [14] S. Chaudhuri. 1998. An Overview of Query Optimization in Relational Systems. In *Proceedings of the ACM Symposium on Principles of Database Systems*. 34–43. <https://doi.org/10.1145/275487.275492>
- [15] S. Chaudhuri, V. R. Narasayya, and R. Ramamurthy. 2009. Exact Cardinality Query Optimization for Optimizer Testing. *PVLDB* 2, 1 (2009), 994–1005. <https://doi.org/10.14778/1687627.1687739>
- [16] J. Cheney, L. Chiticariu, and W.-C. Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474. <https://doi.org/10.1561/19000000006>
- [17] Y. Chronis, Y. Wang, Y. Gan, S. Abu-El-Haija, C. Lin, C. Binnig, and F. Özcan. 2024. CardBench: A Benchmark for Learned Cardinality Estimation in Relational Databases. *CoRR* abs/2408.16170 (2024). [arXiv:2408.16170](https://arxiv.org/abs/2408.16170)
- [18] S. Chu, B. Murphy, J. Roesch, A. Cheung, and D. Suciu. 2018. Axiomatic Foundations and Algorithms for Deciding Semantic Equivalences of SQL Queries. *PVLDB* 11, 11 (2018), 1482–1495. <https://doi.org/10.14778/3236187.3236200>
- [19] S. Chu, C. Wang, K. Weitz, and A. Cheung. 2017. Cosette: An Automated Prover for SQL. In *Proceedings of the Conference on Innovative Data Systems Research*. 1–10. <https://www.cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf>
- [20] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (1970), 377–387. <https://doi.org/10.1145/362384.362685>
- [21] R. L. Cole and G. Graefe. 1994. Optimization of Dynamic Query Evaluation Plans. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 150–160. <https://doi.org/10.1145/191839.191872>
- [22] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *Proceedings of the ACM Symposium on Cloud Computing*. 143–154. <https://doi.org/10.1145/1807128.1807152>
- [23] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 215–226. <https://doi.org/10.1145/2882903.2903741>
- [24] Jérôme Darmont, Fadila Bentayeb, and Omar Boussaïd. 2007. Benchmarking Data Warehouses. *International Journal of Business Intelligence and Data Mining* 2, 1 (2007), 79–104. <https://doi.org/10.1504/IJBIDM.2007.012947>
- [25] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *PVLDB* 17, 8 (2024), 1925–1938. <https://doi.org/10.14778/3659437.3659448>
- [26] A. Deshpande, Z. Ives, and V. Raman. 2007. Adaptive Query Processing. *Foundations and Trends in Databases* 1, 1 (2007), 1–140. <https://doi.org/10.1561/19000000001>
- [27] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, Magda Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. 2015. The BigDAWG Polystore System. *SIGMOD Record* 44, 2 (2015), 11–16. <https://doi.org/10.1145/2814710.2814713>
- [28] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC Social Network Benchmark: Interactive Workload. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 619–630. <https://doi.org/10.1145/2723372.2742786>
- [29] Cesar A. Galindo-Legaria and Arnon Rosenthal. 1997. Outerjoin Simplification and Reordering for Query Optimization. *ACM Transactions on Database Systems* 22, 1 (1997), 43–74. <https://doi.org/10.1145/244810.244812>
- [30] G. Graefe. 1990. Encapsulation of Parallelism in the Volcano Query Processing System. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 102–111. <https://doi.org/10.1145/93597.98720>
- [31] G. Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Engineering Bulletin* 18, 3 (1995), 19–29. <https://www.sigmod.org/publications/dblp/db/journals/debu/Graefe95a.html>
- [32] G. Graefe and W. J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. In *Proceedings of the IEEE International Conference on Data Engineering*. 209–218. <https://doi.org/10.1109/ICDE.1993.344061>
- [33] T. J. Green. 2011. Containment of Conjunctive Queries on Annotated Relations. *Theory of Computing Systems* 49, 2 (2011), 429–459. <https://doi.org/10.1007/s00224-011-9327-6>
- [34] T. J. Green, G. Karvounarakis, and V. Tannen. 2007. Provenance Semirings. In *Proceedings of the ACM Symposium on Principles of Database Systems*. 31–40. <https://doi.org/10.1145/1265530.1265535>
- [35] Anurag Gupta, Deepak Agarwal, Derek Tan, Jakub Kulesza, Rahul Pathak, Stefano Stefani, and Vidhya Srinivasan. 2015. Amazon Redshift and the Case for Simpler Data Warehouses. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1917–1923. <https://doi.org/10.1145/2723372.2742795>
- [36] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *PVLDB* 15, 4 (2021), 752–765. <https://doi.org/10.14778/3503585.3503586>
- [37] B. Hilprecht, A. Schmidt, M. Kulesza, A. Molina, K. Kersting, and C. Binnig. 2020. DeepDB: Learn from Data, Not from Queries! *PVLDB* 13, 7 (2020), 992–1005. <https://doi.org/10.14778/3384345.3384349>
- [38] Y. E. Ioannidis. 1996. Query Optimization. *Comput. Surveys* 28, 1 (1996), 121–123. <https://doi.org/10.1145/234313.234367>
- [39] Z. G. Ives, D. Florescu, M. Friedman, A. Levy, and D. S. Weld. 1999. An Adaptive Query Execution System for Data Integration. *ACM SIGMOD Record* 28, 2 (1999), 299–310. <https://doi.org/10.1145/304181.304209>
- [40] David Justen, Daniel Ritter, Campbell Fraser, Andrew Lamb, Allison Lee, Thomas Bodner, Mhd Yamen Haddad, Steffen Zeuch, Volker Markl, and Matthias Boehm. 2024. POLAR: Adaptive and Non-invasive Join Order Selection via Plans of Least Resistance. *PVLDB* 17, 6 (2024), 1350–1363. <https://doi.org/10.14778/3648160.3648175>
- [41] N. Kabra and D. J. DeWitt. 1998. Efficient Mid-Query Re-Optimization of Sub-Optimal Query Execution Plans. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 106–117. <https://doi.org/10.1145/276304.276315>

- [42] Alfons Kemper and Thomas Neumann. 2011. HyPer: A Hybrid OLTP & OLAP Main Memory Database System Based on Virtual Memory Snapshots. In *Proceedings of the IEEE International Conference on Data Engineering*. 195–206. <https://doi.org/10.1109/ICDE.2011.5767867>
- [43] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *Proceedings of the Conference on Innovative Data Systems Research*. 1–11. <https://www.cidrdb.org/cidr2019/papers/p101-kipf-cidr19.pdf>
- [44] Marcel Kornacker, Alexander Behm, Victor Bittorf, Taras Bobrovitsky, Casey Ching, Alan Choi, Justin Erickson, Martin Grund, Daniel Hecht, Matthew Jacobs, Ishaan Joshi, Lenni Kuff, Dileep Kumar, Alex Leblang, Nong Li, Ippokratis Pandis, Henry Robinson, David Rorke, Silvius Rus, John Russell, Dimitris Tsirgiannis, Skye Wanderman-Milne, and Michael Yoder. 2015. Impala: A Modern, Open-Source SQL Engine for Hadoop. In *Proceedings of the Conference on Innovative Data Systems Research*. https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper28.pdf
- [45] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2015. How Good Are Query Optimizers, Really? *PVLDB* 9, 3 (2015), 204–215. <https://doi.org/10.14778/2850583.2850594>
- [46] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2018. Query Optimization Through the Looking Glass, and What We Found Running the Join Order Benchmark. *Vldb Journal* 27, 5 (2018), 643–668. <https://doi.org/10.1007/s00778-017-0480-7>
- [47] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1275–1288. <https://doi.org/10.1145/3448016.3452838>
- [48] R. Marcus and O. Papaemmanouil. 2019. Neo: A Learned Query Optimizer. *PVLDB* 12, 11 (2019), 1705–1718. <https://doi.org/10.14778/3342263.3342644>
- [49] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilindzic. 2004. Robust Query Processing Through Progressive Optimization. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 659–670. <https://doi.org/10.1145/1007568.1007642>
- [50] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. 2010. Dremel: Interactive Analysis of Web-Scale Datasets. *PVLDB* 3, 1-2 (2010), 330–339. <https://doi.org/10.14778/1920841.1920886>
- [51] G. Moerkotte and T. Neumann. 2006. Analysis of Two Existing and One New Dynamic Programming Algorithm for the Generation of Optimal Bushy Join Trees without Cross Products. In *Proceedings of the International Conference on Very Large Data Bases*. 930–941. <https://dblp.org/rec/conf/vldb/MoerkotteN06.html>
- [52] Raghunath Othayoth Nambiar and Meikel Poess. 2006. The Making of TPC-DS. In *Proceedings of the International Conference on Very Large Data Bases*. 1049–1058. <https://www.vldb.org/conf/2006/p1049-othayoth.pdf>
- [53] T. Neumann. 2011. Efficiently Compiling Efficient Query Plans for Modern Hardware. *PVLDB* 4, 9 (2011), 539–550. <https://doi.org/10.14778/2002938.2002940>
- [54] P. O’Neil, E. O’Neil, X. Chen, and S. Revilak. 2009. The Star Schema Benchmark and Augmented Fact Table Indexing. In *Proceedings of the TPC Technology Conference on Performance Evaluation and Benchmarking*. 237–252. https://doi.org/10.1007/978-3-642-10424-4_17
- [55] Andrew Pavlo, Erik Paulson, Alexander Rasin, Daniel J. Abadi, David J. DeWitt, Samuel Madden, and Michael Stonebraker. 2009. A Comparison of Approaches to Large-Scale Data Analysis. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 165–178. <https://doi.org/10.1145/1559845.1559865>
- [56] Meikel Poess and Chris Floyd. 2000. New TPC Benchmarks for Decision Support and Web Commerce. *ACM SIGMOD Record* 29, 4 (2000), 64–71. <https://doi.org/10.1145/369275.369291>
- [57] M. Raasveldt and H. Muehleisen. 2019. DuckDB: An Embeddable Analytical Database. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- [58] M. Rigger and Z. Su. 2020. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1140–1152. <https://doi.org/10.1145/3368089.3409710>
- [59] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Synthesis. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 667–682. <https://www.usenix.org/conference/osdi20/presentation/rigger>
- [60] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 23–34. <https://doi.org/10.1145/582095.582099>
- [61] Jeff Shute, Radek Vingralek, Bart Samwel, Ben Handy, Chad Whipkey, Eric Rollins, Mircea Oancea, Kyle Littlefield, David Menestrina, Stephan Ellner, John Cieslewicz, Ian Rae, Traian Stancescu, and Himani Apte. 2013. F1: A Distributed SQL Database That Scales. *PVLDB* 6, 11 (2013), 1068–1079. <https://doi.org/10.14778/2536222.2536232>
- [62] Mohamed A. Soliman, Lyublena Antova, Venkatesh Raghavan, Amr El-Helw, Zhongxian Gu, Entong Shen, George C. Caragea, Carlos Garcia-Alvarado, Foyzur Rahman, Michalis Petropoulos, Florian Waas, Sivaramakrishnan Narayanan, Konstantinos Krikellias, and Rhonda Baldwin. 2014. Orca: A Modular Query Optimizer Architecture for Big Data. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 337–348. <https://doi.org/10.1145/2588555.2595637>
- [63] M. Steinbrunn, G. Moerkotte, and A. Kemper. 1997. Heuristic and Randomized Optimization for the Join Ordering Problem. *Vldb Journal* 6, 3 (1997), 191–208. <https://doi.org/10.1007/s007780050040>
- [64] Michael Stonebraker, Daniel J. Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Samuel Madden, Elizabeth J. O’Neil, Patrick E. O’Neil, Alexander Rasin, Nga Tran, and Stanley B. Zdonik. 2005. C-Store: A Column-Oriented DBMS. In *Proceedings of the International Conference on Very Large Data Bases*. 553–564. <https://doi.org/10.5555/1083592.1083658>
- [65] Ashish Thusoo, Joydeep Sen Sarma, Namit Jain, Zheng Shao, Prasad Chakka, Suresh Anthony, Hao Liu, Pete Wyckoff, and Raghobham Murthy. 2009. Hive: A Warehousing Solution over a Map-Reduce Framework. *PVLDB* 2, 2 (2009), 1626–1629. <https://doi.org/10.14778/1687553.1687609>
- [66] S. Wang, S. Pan, and A. Cheung. 2024. QED: A Powerful Query Equivalence Decoder for SQL. *PVLDB* 17, 11 (2024), 3602–3614. <https://doi.org/10.14778/3681954.3682024>
- [67] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, and S. Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27. <https://doi.org/10.1145/3588721>
- [68] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, and I. Stoica. 2019. Deep Unsupervised Cardinality Estimation. *PVLDB* 13, 3 (2019), 279–292. <https://doi.org/10.14778/3368289.3368294>
- [69] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation*. 15–28. <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>
- [70] Marcin Zukowski, Mark van de Wiel, and Peter Boncz. 2012. Vectorwise: A Vectorized Analytical DBMS. In *Proceedings of the IEEE International Conference on Data Engineering*. 1349–1350. <https://doi.org/10.1109/ICDE.2012.148>